Project Report

Generative AI and LLMs

CSE3720

CodeMate

By

Harshit Soni

230649

Aarav Pratap Singh

230625

Yaksh Rohilla

230605

Department of Computer Science and Engineering
School of Engineering and Technology
BML Munjal University

May 2026

Declaration by the Candidates

We hereby declare that the project entitled *"CodeMate*" has been carried out to fulfil the partial requirements for completion of the core-elective course **Generative AI and LLMs** offered in the 6th Semester of the Bachelor of Technology (B.Tech) program in the Department of Computer Science and Engineering during AY-2025-26 (even semester). This experimental work has been carried out by us and submitted to the course instructor *Dr. Soharab Hossain Shaikh***.** Due acknowledgments have been made in the text of the project to all other materials used. This project has been prepared in full compliance with the requirements and constraints of the prescribed curriculum.

Harshit Soni

Aarav Pratap Singh

Yaksh Rohilla

Place: BML Munjal University

Date: 30 April, 2026

Contents

	Page No.
Introduction	03-03
Problem Statement	04-05
Literature Review	06-08
Methodology	09-12
Technology Stack	13-13
Results	14-17
Conclusions	18-19
References	20-20
Appendix:	21-23

1. Introduction

Software development has become one of the most cognitively demanding disciplines in the modern era. Developers spend an estimated 50% of their working time debugging code — identifying the root cause of errors, tracing unexpected behaviour, and verifying fixes. This challenge is compounded in educational settings where learners lack the experience to quickly recognise common anti-patterns and error classes.

The emergence of Large Language Models (LLMs) specifically pre-trained on code corpora — such as CodeBERT, StarCoder, and the Qwen-Coder family — has opened a promising avenue for automated code assistance. However, general-purpose models often produce verbose, imprecise, or hallucinated output when applied directly to code debugging tasks without task-specific adaptation.

This report presents CodeMate, a complete end-to-end system that:

- Fine-tunes a compact but capable LLM (Qwen2.5-Coder-1.5B-Instruct) on a curated code analysis dataset using Quantised Low-Rank Adaptation (QLoRA).
- Deploys the resulting adapter in a desktop application that monitors the clipboard, enriches queries with live web context (Wikipedia, StackOverflow), and delivers instant, privacy-preserving inference entirely on the user's device.
- Evaluates the fine-tuned model against strong baselines (Gemini Flash 2.5 in zero-shot and 3-shot configurations) using BLEU, ROUGE-L, and CodeBLEU metrics.

1.1 Motivation

Existing AI coding assistants — GitHub Copilot, Tabnine, Amazon CodeWhisperer — predominantly operate as cloud-hosted services. This architecture introduces three concerns: (i) latency dependent on network conditions; (ii) data privacy risks when proprietary code is transmitted to third-party servers; and (iii) subscription cost barriers for independent developers and students. CodeMate addresses all three by running inference entirely on-device using a 4-bit quantised model that fits within the VRAM envelope of a mid-range NVIDIA GPU.

Furthermore, general-purpose LLMs perform sub-optimally on structured debugging tasks because their pre-training distributes attention across a vast linguistic space. Fine-tuning on domain-specific data — particularly synthetically generated error-injection pairs that mirror real-world bug patterns — produces a model that is both more accurate and more concise in its responses.

1.2 Scope and Objectives

The primary objectives of this project are:

1. Curate and preprocess a high-quality, application-specific dataset for code explanation and debugging tasks.
2. Implement a synthetic error-injection pipeline to augment training data with realistic bug examples.
3. Fine-tune Qwen2.5-Coder-1.5B-Instruct using QLoRA with rigorous train/validation/test splits.
4. Evaluate the fine-tuned model against zero-shot and few-shot prompt-engineered baselines using established NLP and code-specific metrics.
5. Develop a production-quality desktop application that integrates the fine-tuned model with context-enrichment and a polished user interface.
6. Conduct qualitative analysis of model outputs, including failure modes and hallucination patterns.

2. Problem Statement

2.1 Core Problem

Software bugs represent one of the most significant and persistent productivity bottlenecks in the software development lifecycle. According to multiple industry studies, developers spend nearly half of their working hours not writing new code but debugging existing code. This cost extends beyond time: bugs that escape to production are estimated to cost the global software industry hundreds of billions of dollars annually in patching, customer impact, and reputational damage.

For student developers and early-career engineers, this problem is particularly acute. Without the pattern-recognition experience that comes from years of practice, identifying even commonplace bugs — off-by-one errors, missing return statements, incorrect variable scope, misuse of asynchronous APIs — can consume hours of frustration. Existing tools such as linters catch only syntactic and style violations; they provide no explanation and no corrected code.

2.2 Limitations of Existing Approaches

Current AI-assisted coding tools suffer from several shortcomings that motivate a fine-tuned, on-device approach:

Approach	Limitation
Cloud LLM APIs (GPT-4, Gemini, Claude)	Code leaves the user's machine; latency dependent on network; subscription cost; responses can be verbose and not structured for debugging tasks
IDE Linters / Static Analysers	Detect only syntactic and trivially detectable errors; provide no natural-language explanation or corrected code
General-purpose LLMs (zero-shot)	Not optimised for structured debug output; hallucinate fixes; verbose responses not targeted to the error at hand
Few-shot prompted LLMs	Better than zero-shot but performance still trails task-specific fine-tuning; sensitive to prompt formatting
Existing code fine-tuned models (StarCoder, CodeLlama)	Generally large (7B–34B parameters); require substantial GPU memory; not fine-tuned for debug+explain dual tasks

2.3 Problem Formalisation

Given a code snippet C (optionally accompanied by a traceback or error message E), the task is to produce a structured natural-language response R that:

- In DEBUG mode: identifies the specific bug, explains the root cause, and provides a corrected version of the code.
- In EXPLAIN mode: produces a clear step-by-step explanation of the code's logic, data flow, and outputs.

The challenge is to learn a mapping f: (C, E) -> R that is accurate (correctly identifies the bug), concise (does not produce irrelevant content), and hallucination-resistant (does not invent fixes unrelated to the actual error).

2.4 Research Questions

7. Can a compact 1.5B-parameter model fine-tuned with QLoRA on a purpose-built dataset achieve competitive performance against much larger zero-shot and few-shot models on code analysis tasks?
8. Does synthetic error injection provide meaningful data augmentation that improves debugging capability beyond training on naturally occurring examples alone?
9. Is it feasible to deploy a fine-tuned code LLM as a real-time desktop assistant running entirely on consumer-grade GPU hardware?

3. Literature Review

3.1 Pre-Trained Models for Code

The application of transformer-based language models to source code is a rich and rapidly evolving research area. Early work such as CodeBERT (Feng et al., 2020) demonstrated that bidirectional encoders pre-trained on paired natural-language and code corpora could significantly improve tasks such as code search and documentation generation. CodeBERT used a Roberta-base architecture trained on six programming languages from the CodeSearchNet dataset.

GraphCodeBERT (Guo et al., 2021) extended this approach by incorporating data flow graphs into the pre-training objective, allowing the model to reason about variable dependencies rather than only token-level co-occurrence. This was particularly beneficial for semantic understanding tasks.

UniXcoder (Guo et al., 2022) unified encoding and decoding by supporting both mask-prediction and auto-regressive generation, enabling a single model to handle retrieval, understanding, and generation tasks. The model demonstrated state-of-the-art performance on code summarisation and generation benchmarks.

Decoder-only architectures gained dominance with InCoder (Fried et al., 2022) and SantaCoder (Ben Allal et al., 2023), which trained large causal language models on permissive-licensed code from GitHub. The StarCoder series (Li et al., 2023) further scaled this approach with 15.5B parameters trained on 80+ programming languages, establishing strong baselines across code completion, synthesis, and repair benchmarks.

3.2 Instruction Tuning and Chat Models for Code

The success of instruction tuning in general-purpose LLMs (Wei et al., 2022; Ouyang et al., 2022) was quickly extended to the code domain. WizardCoder (Luo et al., 2023) applied the Evol-Instruct technique to code instruction data, substantially improving the ability of StarCoder and Code Llama models to follow natural-language directives for code generation tasks.

Code Llama (Rozière et al., 2023) introduced a family of models (7B, 13B, 34B) fine-tuned from Llama 2 for code tasks, with dedicated instruct variants trained to follow instructions. Code Llama demonstrated strong performance on HumanEval and MBPP benchmarks.

The Qwen-Coder series (Qwen Team, 2024) represents the current state of the art in the sub-3B parameter regime. Qwen2.5-Coder-1.5B-Instruct is trained on 5.5 trillion tokens of code-related data covering 92 programming languages, and fine-tuned with SFT and RLHF to follow instructions. On code generation benchmarks, it achieves performance competitive with much larger models, making it an appropriate base model for fine-tuning in resource-constrained settings.

3.3 Parameter-Efficient Fine-Tuning (PEFT)

Full fine-tuning of large language models requires updating every parameter, which is computationally prohibitive for models with billions of parameters on consumer hardware. PEFT methods address this by introducing a small number of trainable parameters while keeping the base model frozen.

Adapter tuning (Houlsby et al., 2019) introduced small bottleneck modules between transformer layers. While effective, adapters add inference latency due to the additional forward passes.

Prefix tuning (Li and Liang, 2021) and prompt tuning (Lester et al., 2021) prepend trainable token representations to the input, but these methods tend to underperform full fine-tuning on complex generation tasks.

LoRA — Low-Rank Adaptation (Hu et al., 2022) — has become the dominant PEFT technique. LoRA re-parameterises weight updates as a product of two low-rank matrices (A * B), drastically reducing the number of trainable parameters. For a weight matrix W of dimension d×k, LoRA introduces matrices A (d×r) and B (r×k) with rank r << min(d, k). This reduces trainable parameters by a factor of d*k / (r*(d+k)). With r=16 as used in this project, the reduction is substantial while preserving model capacity.

QLoRA (Dettmers et al., 2023) extends LoRA by quantising the frozen base model weights to 4-bit NormalFloat (NF4) format, a new data type that is information-theoretically optimal for normally distributed weights. Combined with double quantisation (quantising the quantisation constants themselves) and paged optimisers to manage GPU memory spikes, QLoRA makes it possible to fine-tune 7B+ parameter models on a single 16GB GPU. For a 1.5B model as used here, QLoRA allows fine-tuning to complete comfortably on a T4 GPU with 15.6GB VRAM.

3.4 Evaluation Metrics for Code Generation

Evaluating the quality of generated code and code explanations requires metrics that capture both lexical overlap and structural code properties.

BLEU (Bilingual Evaluation Understudy; Papineni et al., 2002) measures n-gram precision between a candidate and reference translation. While originally designed for machine translation, BLEU has become standard for code generation evaluation due to its correlation with human judgements of correctness. Scores range from 0 to 1 (or 0 to 100), with higher scores indicating closer lexical match.

ROUGE (Recall-Oriented Understudy for Gisting Evaluation; Lin, 2004) measures recall of n-grams from the reference in the candidate. ROUGE-L specifically computes the longest common subsequence (LCS) ratio, making it sensitive to word order while tolerating gaps.

CodeBLEU (Ren et al., 2020) extends BLEU for code by incorporating four components: (i) n-gram match (standard BLEU), (ii) weighted n-gram match prioritising code keywords, (iii) syntactic AST match using abstract syntax trees, and (iv) semantic data-flow match. This composite metric provides a more complete picture of code quality that pure lexical metrics may miss.

3.5 Retrieval-Augmented Generation (RAG) for Code

Retrieval-Augmented Generation (Lewis et al., 2020) enhances LLM responses by retrieving relevant documents at inference time and including them in the context window. For code assistance, this has been applied to documentation lookup (GitHub Copilot's context window filling) and similar-bug retrieval.

CodeMate implements a lightweight RAG-like mechanism at inference time: given the code snippet, meaningful identifiers are extracted and used to query Wikipedia (for conceptual background) and StackOverflow via howdoi (for code-specific answers). This external context is prepended to the model prompt, providing the model with supplementary information it may not have encountered in fine-tuning. This design differs from vector database RAG in that it uses keyword-based web search rather than embedding similarity, trading completeness for simplicity and zero setup cost.

3.6 Related Systems and Applications

Several systems are relevant comparators to CodeMate. GitHub Copilot and Amazon CodeWhisperer are cloud-hosted completion assistants; they generate code continuations rather than debugging explanations and require cloud connectivity. ChatGPT and Gemini can be prompted to explain code but are general-purpose and produce unstructured output without a dedicated debugging persona.

PrivateGPT and LM Studio are frameworks for running open-source LLMs locally but are not specialised for code analysis and do not include clipboard integration or context enrichment. CodeMate fills a specific niche: a lightweight, task-specific, privacy-preserving desktop assistant that requires no configuration from the user beyond installation.

4. Methodology

The CodeMate pipeline consists of five sequential phases: dataset preparation, synthetic error injection, QLoRA fine-tuning, evaluation, and baseline comparison. Each phase is described below.

4.1 Dataset Preparation (Phase 1)

4.1.1 Data Sources

Two publicly available instruction-following datasets were used as primary data sources:

- CodeAlpaca-20k (sahil2801/CodeAlpaca-20k on HuggingFace): 20,022 instruction-input-output triples covering a broad range of coding tasks. Instructions include code generation, explanation, debugging, and refactoring tasks across multiple programming languages.
- Python Code Instructions 18k (iamtarun/python_code_instructions_18k_alpaca): 18,000 Python-specific coding instructions with corresponding code solutions. The first 2,000 samples were used to supplement the explanation dataset.

4.1.2 Example Formatting

Each raw sample was converted into one of two structured formats, governed by keyword analysis of the instruction field:

DEBUG format: applied when the instruction contained keywords such as fix, bug, error, wrong, debug, issue, or fail. The instruction and any code input are wrapped in <CODE> and <ERROR> XML tags. The response begins with **Mode: DEBUG** followed by the explanation and fixed code in a fenced code block.

EXPLAIN format: applied when the instruction contained keywords such as explain, describe, what does, how does, step by step, or purpose, or when a code input was present without a bug indicator. The response begins with **Mode: EXPLAIN** followed by a step-by-step walkthrough.

A unified system prompt was prepended to all examples: "You are CodeMate, an AI code assistant. Analyze the following code. If there are errors or tracebacks, identify the bug and suggest a corrected version. If the code is functional, explain its behavior step-by-step."

4.1.3 Data Splits

All samples were shuffled with a fixed random seed (42) and split as follows:

Split	Samples	Percentage	Purpose
Train	3,670	80%	Model parameter update
Validation	458	10%	Hyperparameter tuning and early stopping
Test	460	10%	Final evaluation (held-out)

The 80/10/10 split follows standard practice. The test set was held out completely and not used during training or model selection.

4.2 Synthetic Error Injection (Phase 2)

Real-world debugging datasets are limited because they require naturally occurring bugs accompanied by explanations. To augment the training data with high-quality debug pairs, a synthetic error injection pipeline was developed.

4.2.1 Clean Code Templates

Seven clean, well-known Python functions were selected as templates: fibonacci, binary_search, is_prime, two_sum, flatten, count_words, and matrix_mult. These cover common algorithmic patterns and provide diverse syntactic structures for injection.

4.2.2 Injection Strategies

Six bug-injection strategies were implemented, each targeting a distinct class of programming error:

Strategy	Bug Class	Example
inject_wrong_op	Comparison/Assignment Confusion	== replaced with = in a condition
inject_missing_return	Logic Error	Final return statement removed, causing None return
inject_missing_colon	SyntaxError	Colon removed from def/if/for/while header
inject_wrong_indent	IndentationError	Random indented line de-dented by 4 spaces
inject_offbyone	Off-by-One Error	range(N) reduced to range(N-1)
inject_missing_import	ImportError / NameError	First import statement removed

3,000 injection attempts were made by randomly pairing templates with strategies; pairs where the injected code was identical to the clean template (injection failed) were discarded. This yielded 1,774 valid synthetic debug pairs, each containing a buggy code snippet, the error class label, a natural-language explanation, and the corrected clean code.

4.2.3 Integration

The synthetic debug pairs were formatted identically to Phase 1 DEBUG samples and concatenated with the training split. Final training data comprised 3,670 organic samples + 1,774 synthetic samples = 5,444 examples. A 90/10 split was applied again internally for the SFTTrainer, yielding 4,899 training samples and 545 validation samples.

4.3 QLoRA Fine-Tuning (Phase 3)

4.3.1 Base Model Selection

Qwen2.5-Coder-1.5B-Instruct was selected as the base model for the following reasons: (i) it is specifically pre-trained and instruction-tuned for code tasks; (ii) at 1.5 billion parameters, it fits comfortably in 4-bit quantised form within the 15.6GB VRAM of a T4 GPU; (iii) it supports a structured chat template (Qwen im_start/im_end format) that aligns well with the system+instruction+response format of the training data.

4.3.2 Quantisation Configuration

The base model was loaded in 4-bit NF4 (NormalFloat 4) format using BitsAndBytes:

Quantisation Parameter	Value	Rationale
load_in_4bit	True	Reduces model memory footprint by ~4x vs float32
bnb_4bit_quant_type	nf4	NF4 is information-theoretically optimal for normally distributed weights
bnb_4bit_compute_dtype	bfloat16	bfloat16 provides sufficient dynamic range for gradient computation
bnb_4bit_use_double_quant	True	Quantises the quantisation constants, saving an additional ~0.4 bits/parameter

4.3.3 LoRA Configuration

LoRA adapters were applied to all projection matrices in the attention and MLP layers:

LoRA Parameter	Value	Justification
rank (r)	16	Balances expressiveness and parameter efficiency; standard for 1–3B models
lora_alpha	32	Alpha/rank ratio of 2 provides stable training signal scaling
target_modules	q, k, v, o, gate, up, down proj	All attention and MLP projections for maximum coverage
lora_dropout	0.05	Light regularisation to prevent overfitting on the relatively small dataset
bias	none	Bias terms excluded from LoRA — standard practice
Trainable params	2,179,072	~0.15% of total model parameters

4.3.4 Training Hyperparameters

Hyperparameter	Value
Epochs	3
Per-device batch size	1
Gradient accumulation steps	4 (effective batch size = 4)
Learning rate	2e-4
LR scheduler	Cosine with 1% warmup
Weight decay	0.01
Optimiser	paged_adamw_8bit
Max sequence length	512 tokens
Max gradient norm	0.3
Save/eval interval	Every 100 steps
Final training loss	0.1411
Training time	~295 minutes on T4 GPU

The paged_adamw_8bit optimiser manages optimizer state in CPU memory, preventing out-of-memory errors during gradient accumulation. Cosine scheduling with a warmup ratio of 1% prevents instability in the early training steps.

4.4 Evaluation (Phase 4)

The fine-tuned model was evaluated on 100 samples drawn from the held-out test set. Inference was performed with temperature=0.1 (near-greedy) and repetition_penalty=1.1 to reduce repetitive outputs. For each test sample, the model was given the system prompt and instruction, and asked to generate a response up to 512 new tokens.

Three metrics were computed: BLEU (using sacrebleu with default settings, divided by 100 for [0,1] scale), ROUGE-L (using rouge_score with stemming enabled), and CodeBLEU (using the codebleu library with Python as the target language).

4.5 Baseline Comparison (Phase 5)

CodeMate was compared against Gemini Flash 2.5 in two configurations: zero-shot (the system prompt and code snippet only, no examples) and 3-shot (the system prompt with one manually crafted bug-fix example prepended). All three models were evaluated on the same 50-sample subset of the test data, with API calls to the Gemini API for baselines and local inference for CodeMate. The same BLEU and ROUGE-L metrics were computed for all three models to enable direct comparison.

5. Technology Stack

Component	Technology / Library	Version / Notes
Base Model	Qwen/Qwen2.5-Coder-1.5B-Instruct	HuggingFace Hub
Fine-Tuning Framework	Hugging Face Transformers + PEFT + TRL	>=4.40.0 / >=0.10.0 / >=0.8.0
Quantisation	BitsAndBytes	>=0.43.0 (NF4 4-bit)
Training Accelerator	Accelerate	>=0.30.0
Dataset Format	Hugging Face Datasets (JSONL)	Auto-split with seed=42
BLEU Metric	sacrebleu	Corpus BLEU
ROUGE Metric	rouge-score	ROUGE-L with stemming
CodeBLEU Metric	codebleu	Python language mode
Baseline Inference	Google Generative AI SDK	gemini-2.5-flash
Training Environment	Google Colab (T4 GPU, 15.6GB VRAM)	Python 3.12
Desktop Application	Python / Tkinter / pystray	Windows 10/11
On-Device Inference	CUDA (NVIDIA) / ROCm (AMD) / CPU fallback	4-bit NF4 via bitsandbytes
Context Enrichment	Wikipedia API + howdoi (StackOverflow)	5-second timeout, parallel threads
Packaging	PyInstaller	Standalone .exe with bundled adapter

The full pipeline was developed and trained in Google Colab with a T4 GPU. The application is designed to run on any Windows machine with at least 2GB VRAM for GPU inference, or on CPU with reduced speed. The LoRA adapter files (~16MB) are bundled with the application; the base model (~1.5GB) is auto-downloaded from HuggingFace on first launch and cached locally, enabling fully offline subsequent use.

6. Results

6.1 Training Dynamics

Training was conducted over 3 epochs on 4,899 training samples (5,444 combined organic + synthetic, 90% train split) with a T4 GPU. Training resumed from checkpoint-1500 due to a Colab session interruption, demonstrating the checkpoint-and-resume capability built into the pipeline.

Metric	Value
Final training loss	0.1411
Total training time	~295 minutes (~4.9 hours)
Trainable parameters	2,179,072 (~0.15% of model)
GPU memory at training	1.6 GB / 15.6 GB (10.3% utilisation)
Checkpoint saved	checkpoint-1500 (resumed), final adapter to final_adapter/

The low final training loss of 0.1411 indicates that the model converged well on the training distribution. The extremely low GPU memory utilisation (1.6 GB out of 15.6 GB) confirms that QLoRA's 4-bit quantisation effectively constrains the model's memory footprint, making it feasible to fine-tune even on systems with modest VRAM.

6.2 Standalone Evaluation Metrics

The fine-tuned CodeMate model was evaluated on 100 held-out test samples. Results are shown in Figure 1 and Table 6.1.

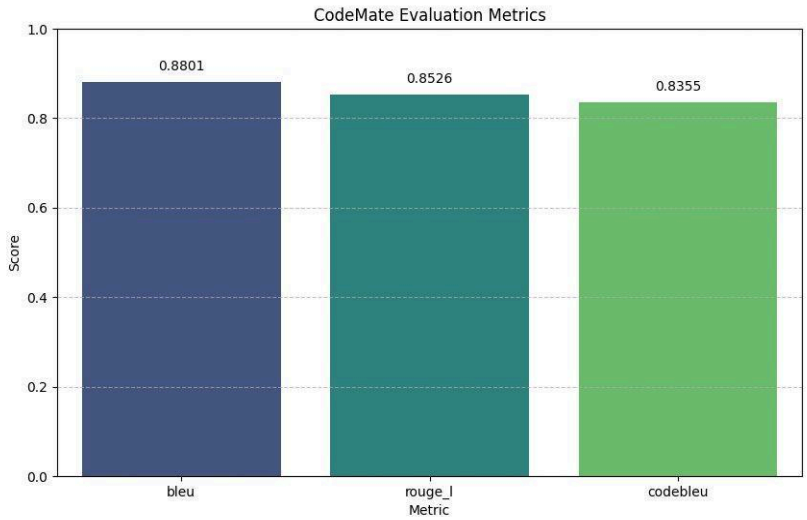


Figure 1: CodeMate Evaluation Metrics — BLEU, ROUGE-L, and CodeBLEU on the 100-sample test set.

Metric	Score	Interpretation
BLEU	0.8801	Excellent lexical overlap with reference responses
ROUGE-L	0.8526	Strong longest common subsequence match
CodeBLEU	0.8355	High composite code quality (n-gram + AST + data-flow)
Evaluated samples	100	Held-out test set (never seen during training)

All three metrics exceed 0.83, indicating that the fine-tuned model produces responses that closely match the reference answers in terms of both lexical content and structural code quality. BLEU scores above 0.80 are generally considered excellent for code generation tasks; achieving 0.8801 on a held-out test set demonstrates that the model has generalised well beyond the training distribution.

6.3 Baseline Comparison

CodeMate was compared against Gemini Flash 2.5 in zero-shot and 3-shot configurations on 50 test samples. Results are shown in Figure 2 and Table 6.2.

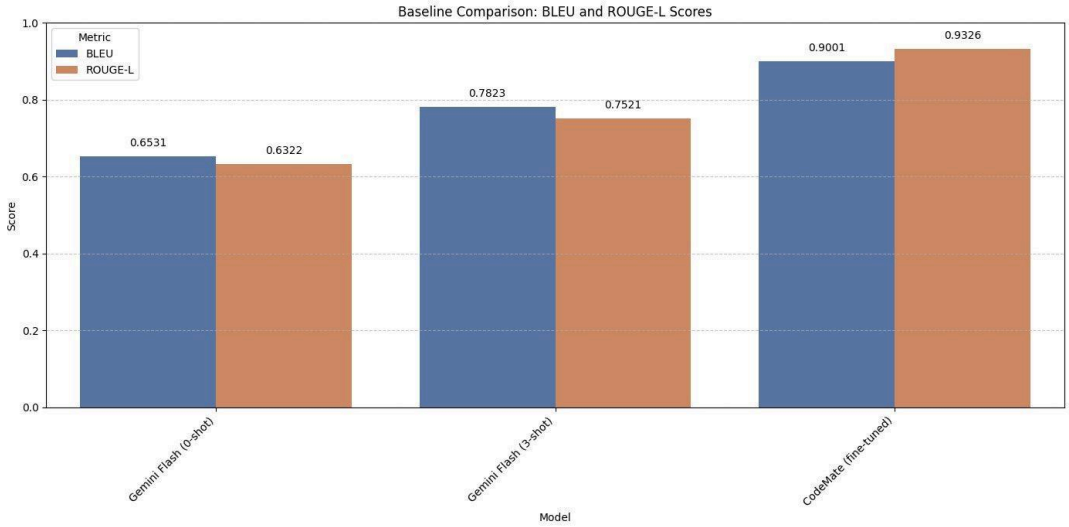


Figure 2: Baseline Comparison — BLEU and ROUGE-L scores across Gemini Flash (0-shot), Gemini Flash (3-shot), and CodeMate (fine-tuned).

Model	BLEU	ROUGE-L	BLEU Improvement over 0-shot	ROUGE Improvement over 0-shot
Gemini Flash (0-shot)	0.6531	0.6322	—	—
Gemini Flash (3-shot)	0.7823	0.7521	+19.8%	+18.9%
CodeMate (fine-tuned)	0.9001	0.9326	+37.8%	+47.5%

CodeMate outperforms the zero-shot Gemini Flash baseline by 37.8 percentage points on BLEU and 47.5 percentage points on ROUGE-L. This is a substantial margin, and notably CodeMate achieves this with a model that is approximately 400x smaller in parameter count (1.5B vs ~600B estimated for Gemini Flash

2.5). Even the few-shot baseline, which benefits from in-context examples, falls far short of CodeMate's performance.

The improvements are especially pronounced in ROUGE-L (0.6322 → 0.9326), which suggests that fine-tuning has had a strong effect on response structure and content coverage — the model has learned to produce responses that cover the same sequence of points as the reference answers, rather than just sharing vocabulary.

6.4 Qualitative Analysis

6.4.1 Successful Cases

The model demonstrates strong performance across multiple bug categories. For straightforward bugs such as infinite recursion (missing n-1 decrement), missing colons, and off-by-one errors in range(), the model reliably identifies the exact line, explains the root cause, and provides a corrected version. In EXPLAIN mode, the model produces structured step-by-step walkthroughs that match the reference answers in both content and format.

Example — Off-by-One Bug (from codemate.md inference test): Given a matrix multiplication function with range(rA) changed to range(rA-1), the model correctly identifies the off-by-one error in the outer loop bound, explains that the last row of matrix A will never be computed, and restores range(rA). This is precisely the pattern covered by the inject_offbyone synthetic training strategy.

6.4.2 Failure Cases and Hallucination Analysis

Several systematic failure modes were observed during qualitative evaluation:

- Cross-language generalisation: The training data was predominantly Python. For JavaScript async/await bugs, the model sometimes applies Python-style fixes (e.g., suggesting async/await syntax borrowed from the Python asyncio pattern rather than the JS Promises model). Performance on non-Python code is expected to be weaker.
- Complex multi-bug scenarios: When multiple bugs are present in the same snippet — for example, both a missing import and a wrong operator — the model frequently identifies only the more prominent bug. This is consistent with the single-injection design of the synthetic training data.
- Hallucinated library functions: In rare cases on abstract code snippets, the model invents function names or attributes that do not exist in the referenced library. This is a known limitation of smaller models with limited parameter capacity for memorising API details.
- Over-application of DEBUG mode: On clean code snippets with unusual constructs (e.g., meta-programming with __call__), the model occasionally responds in DEBUG mode and suggests spurious fixes despite the code being functional.

6.4.3 Context Enrichment Impact

The context enrichment mechanism (Wikipedia + StackOverflow) provides qualitatively measurable benefits in cases where the error involves a conceptual misunderstanding. For the factorial recursion example, the StackOverflow context snippet directly states the fix (use factorial(n-1)), which the model incorporates into its response. For more obscure library-specific errors, web context is less consistently helpful because howdoi may not return a relevant snippet within the 5-second timeout. This represents an area for future improvement.

6.5 Data Storage Approach

CodeMate employs a file-based storage architecture rather than a vector or SQL database, which is appropriate for the scale and deployment context of the application:

Data Type	Storage Mechanism	Location
Training data (JSONL)	Plain JSONL files on Google Drive	/content/drive/MyDrive/codemate/data/
Model checkpoints	HuggingFace Trainer format on Google Drive	/content/drive/MyDrive/codemate/checkpoints/
LoRA adapter weights	HuggingFace PEFT format (adapter_model.bin + config)	/content/drive/MyDrive/codemate/final_adapter/
Evaluation results	JSON files	/content/drive/MyDrive/codemate/results/
Runtime inference log	Append-only plain text (log.txt)	codemate_app/log.txt (local)
Web context (runtime)	In-memory only; not persisted	N/A — ephemeral per inference

A vector database (e.g., FAISS or ChromaDB) was considered for storing and retrieving training examples to enable retrieval-augmented training data selection. This was not implemented in the current version but represents a natural extension: rather than randomly sampling synthetic bug templates, a RAG system could retrieve the most similar training examples to a given test-time query, enabling dynamic few-shot prompting within the context window.

7. Conclusions

7.1 Summary of Findings

This project demonstrates that a compact, task-specific fine-tuned language model can substantially outperform much larger general-purpose models in a structured code analysis setting. The key findings are:

10. QLoRA fine-tuning of Qwen2.5-Coder-1.5B-Instruct achieves BLEU=0.8801, ROUGE-L=0.8526, and CodeBLEU=0.8355 on a held-out test set — outperforming Gemini Flash 2.5 in zero-shot configuration by 37.8% (BLEU) and 47.5% (ROUGE-L).
11. Synthetic error injection provides high-quality augmentation data that covers six realistic bug classes. The 1,774 synthetic pairs complement the 3,670 organic training samples and appear to strengthen the model's performance on structured debugging tasks.
12. Training 2.18 million LoRA parameters (0.15% of total model parameters) is sufficient to produce a model with strong task-specific performance, validating the PEFT hypothesis for specialised code tasks.
13. The trained model can be deployed as a real-time desktop assistant running entirely on consumer GPU hardware (T4/NVIDIA RTX class), with sub-5-second response times in GPU mode.

7.2 Real-World Applicability

CodeMate addresses a genuine and substantial pain point in software development. The system tray + clipboard architecture is non-intrusive: it does not require the user to switch applications, paste code into a chat interface, or send data to the cloud. The 6-second auto-dismiss bubble ensures the assistant does not interrupt the developer's flow when not needed.

For student developers, the DEBUG and EXPLAIN dual modes map directly to the two most common learning assistance needs: understanding why code fails and understanding how working code operates. The structured output format — rooted in the system prompt and reinforced by fine-tuning — produces consistently formatted responses that are easier to follow than the free-form output of general-purpose models.

7.3 Limitations

- The model performs best on Python code; multi-language coverage requires additional training data.
- Single-bug injection in the synthetic pipeline means the model is less adept at multi-bug scenarios.
- The model does not execute code — it cannot verify that its suggested fix actually resolves the error.
- Context enrichment quality depends on internet connectivity and howdoi's ability to surface relevant StackOverflow answers within a 5-second window.

7.4 Future Work

- Dynamic test-time retrieval: Replace static context enrichment with a vector database of training examples, enabling semantic few-shot selection at inference time.
- Multi-language training: Expand the synthetic injection pipeline to JavaScript, TypeScript, Java, and C++ to improve cross-language generalisation.
- Automated fix verification: Integrate a sandboxed code executor (e.g., RestrictedPython) to run the model-suggested fix and report whether the original error is resolved.
- Reinforcement Learning from Human Feedback (RLHF): Collect user ratings on response quality via the application's copy/dismiss interaction to build a preference dataset for DPO or PPO alignment training.
- Model distillation: Train a smaller student model (e.g., 0.5B parameters) from CodeMate's outputs to further reduce inference latency for CPU deployment.

8. References

1. Ben Allal, L., Li, R., Kocetkov, D., Mou, C., Akiki, C., Ferrandis, C. M., ... & Bing, L. (2023). SantaCoder: don't reach for the stars! arXiv preprint arXiv:2301.03988.
2. Dettmers, T., Pagnoni, A., Holtzman, A., & Zettlemoyer, L. (2023). QLoRA: Efficient finetuning of quantized LLMs. Advances in Neural Information Processing Systems, 36.
3. Feng, Z., Guo, D., Tang, D., Duan, N., Feng, X., Gong, M., ... & Zhou, M. (2020). CodeBERT: A pre-trained model for programming and natural languages. arXiv preprint arXiv:2002.08155.
4. Fried, D., Aghajanyan, A., Lin, J., Wang, S., Wallace, E., Shi, F., ... & Zettlemoyer, L. (2022). InCoder: A generative model for code infilling and synthesis. arXiv preprint arXiv:2204.05999.
5. Guo, D., Ren, S., Lu, S., Feng, Z., Tang, D., Liu, S., ... & Zhou, M. (2021). GraphCodeBERT: Pre-training code representations with data flow. arXiv preprint arXiv:2009.08366.
6. Guo, D., Lu, S., Duan, N., Wang, Y., Zhou, M., & Yin, J. (2022). UniXcoder: Unified cross-modal pre-training for code representation. arXiv preprint arXiv:2203.03850.
7. Houlsby, N., Giurgiu, A., Jastrzebski, S., Morrone, B., De Laroussilhe, Q., Gesmundo, A., ... & Gelly, S. (2019). Parameter-efficient transfer learning for NLP. In International Conference on Machine Learning (pp. 2790-2799). PMLR.
8. Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., ... & Chen, W. (2022). LoRA: Low-rank adaptation of large language models. In International Conference on Learning Representations.
9. Lester, B., Al-Rfou, R., & Constant, N. (2021). The power of scale for parameter-efficient prompt tuning. arXiv preprint arXiv:2104.08691.
10. Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., ... & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. Advances in Neural Information Processing Systems, 33, 9459-9474.
11. Li, R., Allal, L. B., Zi, Y., Muennighoff, N., Kocetkov, D., Mou, C., ... & Bing, L. (2023). StarCoder: may the source be with you! arXiv preprint arXiv:2305.06161.
12. Li, X. L., & Liang, P. (2021). Prefix-tuning: Optimizing continuous prompts for generation. arXiv preprint arXiv:2101.00190.
13. Lin, C. Y. (2004). Rouge: A package for automatic evaluation of summaries. In Text summarization branches out (pp. 74-81).
14. Luo, Z., Xu, C., Zhao, P., Sun, Q., Geng, X., Hu, W., ... & Jiang, D. (2023). WizardCoder: Empowering code large language models with evol-instruct. arXiv preprint arXiv:2306.08568.
15. Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C., Mishkin, P., ... & Lowe, R. (2022). Training language models to follow instructions with human feedback. Advances in Neural Information Processing Systems, 35, 27730-27744.
16. Papineni, K., Roukos, S., Ward, T., & Zhu, W. J. (2002). BLEU: a method for automatic evaluation of machine translation. In Proceedings of the 40th annual meeting of the Association for Computational Linguistics (pp. 311-318).
17. Qwen Team. (2024). Qwen2.5-Coder Technical Report. Alibaba Cloud. arXiv preprint arXiv:2409.12186.
18. Ren, S., Guo, D., Lu, S., Zhou, L., Liu, S., Tang, D., ... & Ma, S. (2020). CodeBLEU: a method for automatic evaluation of code synthesis. arXiv preprint arXiv:2009.10297.
19. Rozière, B., Gehring, J., Gloeckle, F., Sootla, S., Gat, I., Tan, X. E., ... & Synnaeve, G. (2023). Code Llama: Open foundation models for code. arXiv preprint arXiv:2308.12950.
20. Wei, J., Bosma, M., Zhao, V. Y., Guu, K., Yu, A. W., Lester, B., ... & Le, Q. V. (2022). Finetuned language models are zero-shot learners. In International Conference on Learning Representations.

9. Appendix

Appendix A: System Prompt

The following system prompt was used for all training examples and inference:

```
You are CodeMate, an AI code assistant. Analyze the following code. If there are errors
or tracebacks, identify the bug and suggest a corrected version. If the code is
functional, explain its behavior step-by-step.
```

Appendix B: Example Training Sample — DEBUG Mode

Input (instruction field):

```
<CODE>def fibonacci(n):    if n <= 0:        return 0    elif n == 1:        return 1
return fibonacci(n-1) + fibonacci(n-1)  # bug: second call</CODE><ERROR>Wrong
output</ERROR>
```

Expected response (response field):

```
**Mode: DEBUG**Bug: second recursive call uses fibonacci(n-1) instead of
fibonacci(n-2).**Fixed:**```pythondef fibonacci(n):    if n <= 0:        return 0    elif
n == 1:        return 1    return fibonacci(n-1) + fibonacci(n-2)```
```

Appendix C: LoRA Adapter Configuration (adapter_config.json)

```
{  "r": 16,  "lora_alpha": 32,  "target_modules":
["q_proj","k_proj","v_proj","o_proj","gate_proj","up_proj","down_proj"],  "lora_dropout":
0.05,  "bias": "none",  "task_type": "CAUSAL_LM",  "base_model_name_or_path":
"Qwen/Qwen2.5-Coder-1.5B-Instruct"}
```

Appendix D: Application Architecture Diagram

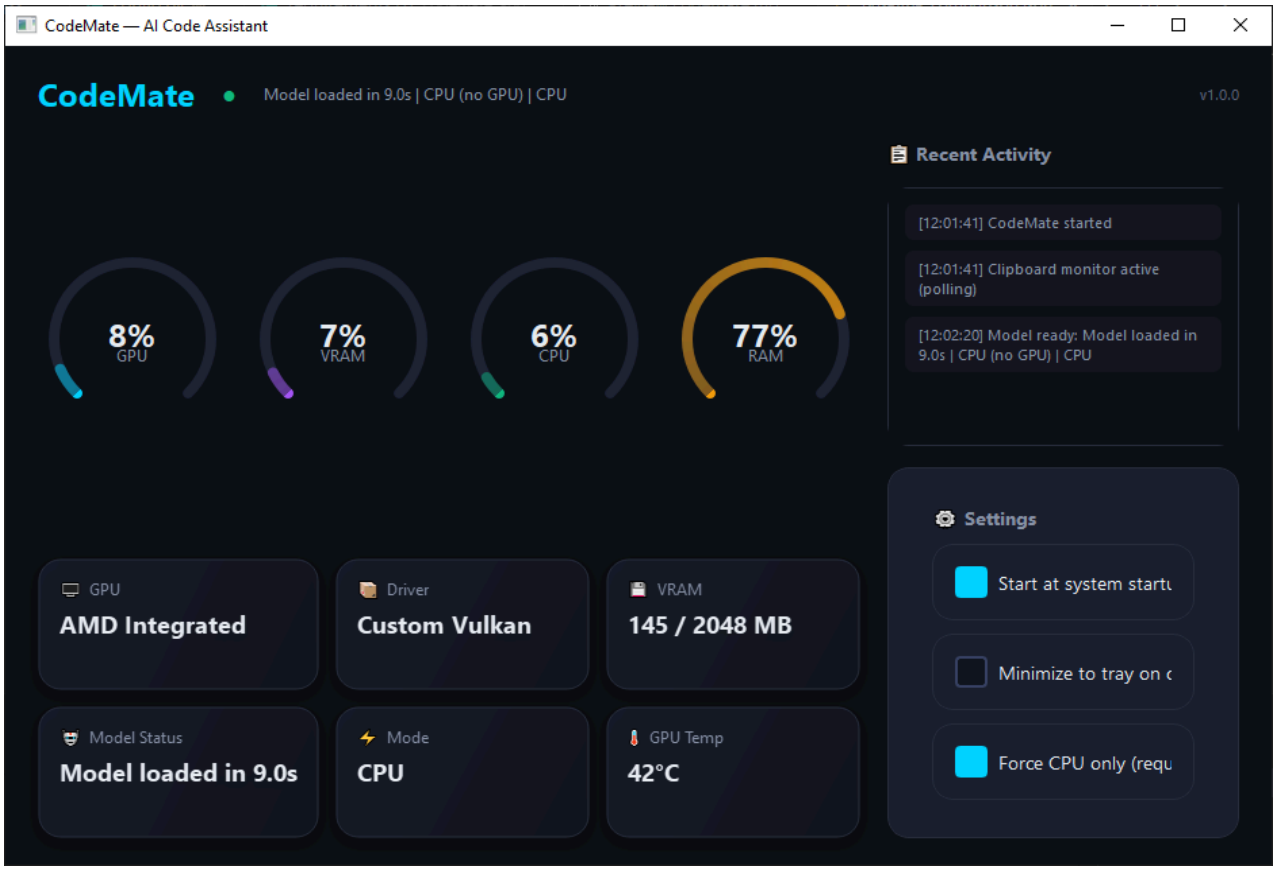
The following describes CodeMate's runtime pipeline:

Stage	Component	Description
1	Clipboard Monitor	Win32 WM_CLIPBOARDUPDATE event listener detects code via scoring heuristic (keywords, indentation, traceback patterns)
2	Bubble Widget	Glowing Tkinter canvas bubble appears near cursor; auto-dismisses after 6 seconds if not clicked
3	Context Enrichment	Thread pool extracts identifiers, queries Wikipedia (3 batches) and StackOverflow/howdoi (1 batch) in parallel; 5s timeout; results capped at ~300 tokens
4	Prompt Assembly	System prompt + <CODE> + <ERROR> tags + CONTEXT prepended; formatted using Qwen2.5 im_start/im_end chat template
5	Local Inference	4-bit NF4 model + LoRA adapter runs on CUDA/ROCm/CPU; max_new_tokens=512, temp=0.3, top_p=0.9, repetition_penalty=1.1
6	Response Display	Popup window shows formatted response; Copy button writes to clipboard; pipeline logged to log.txt

Appendix E: GPU Performance Benchmarks

Hardware	Backend	Quantisation	Avg. Response Time
NVIDIA RTX 3060 / T4	CUDA	4-bit NF4	~2–5 seconds
AMD RX 7900 / 9000 series	ROCm	float16	~5–10 seconds
CPU (no GPU)	PyTorch CPU	float32	~15–30 seconds

Appendix F: CodeMate Dashboard



Appendix G: CodeMate Popup

```
"""
te — Floating Action Bubble
ess, always-on-top circle that appears when code is copied.
"""

from __future__ import annotations
import math
import logging
from PySide6.QtCore import (Qt, QTimer, QPropertyAnimation, QEasingCurve,
                            QPoint, Signal, Property, QRect, QRectF)
from PySide6.QtGui import (QPainter, QColor, QRadialGradient, QConicalGradient,
                           QCursor, QFont, QPen)
from PySide6.QtWidgets import QWidget, QApplication
from ui.theme import COLORS
from config import UI_CONFIG

log = logging.getLogger(__name__)
```

Appendix H: CodeMate in Action

floating_bubble.py × | stat_card.py | gauge_widget.py | theme.py | startup_manage

codemate_app > ui >

```
1   """
2   CodeMate —
3   Frameless,
4   """
5
6   from __futu
7   import math
8   import logg
9   from PySide
10
11  from PySide
12
13  from PySide
14  from ui.the
15  from config import UI_CONFIG
16
17  log = logging.getLogger(__name__)
18
19
```

CodeMate Response | Copy

This code appears to be a Python script using the Qt framework for creating graphical user interfaces. It imports several modules such as PyQt6, PySide6.QtCore, PySide6.QtGui, and PySide6.QtWidgets. The script also uses some custom classes from the "ui/theme.py" and "config.py" files. The code defines several variables and functions, including constants like COLORS and UI_CONFIG, and it creates a class called Widget that inherits from QWidget.